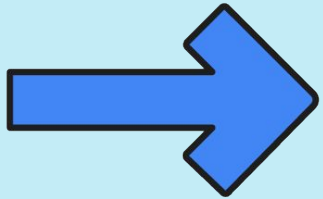
Google Developer Group
Chandigarh University

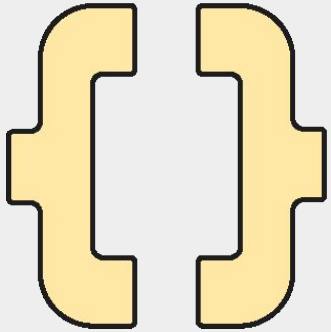
Gemini Study Jams

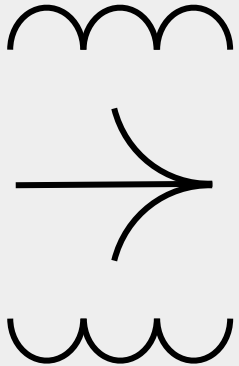
Session 4 - Model Performance and Deployment



What will we do today?

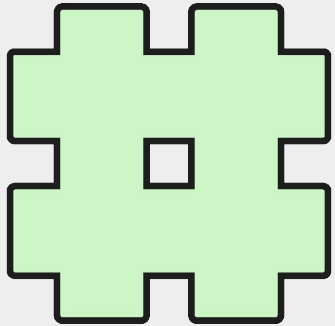
1. When models meet users
2. Offline Metrics vs Real-World behaviour
3. What “good” means?
4. Case Study: Metrics vs Behaviour
5. How evaluation works
6. Metrics & Pitfalls
7. Model to Artifact
8. Failure, Responsibility & Trust
9. Closing Thoughts





1.A. Models meet people

- Users are **not cooperative**
- Inputs are **ambiguous, incomplete, or malformed**
- Prompts **drift** over time
- Scale turns edge cases into the norm



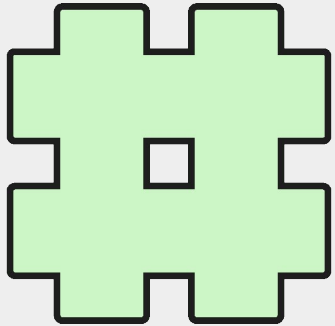
Models, when deployed, **meet people, not data**.

It is natural that a subset of those people use the model in **ways not intended by the developer**.



1.B. Adversarial by accident

- Not **malicious**, but **misaligned**
- **Incentives ≠ developer intent**
- Exploration, **misuse**, and curiosity
- **Rare inputs become common** at scale



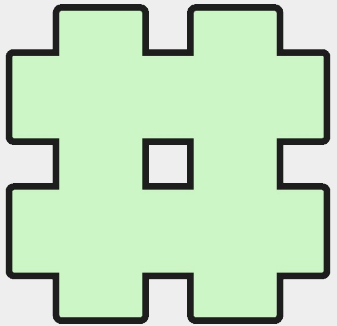
“Adversarial, in this context, **does not mean hostile**. It means **goal-misaligned.**

Scale is an adversarial training process you did not sign up for.



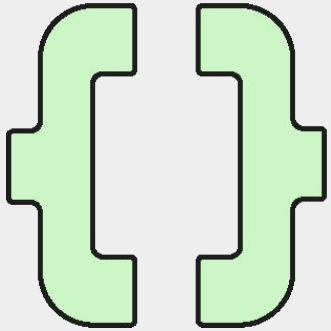
1.C. Experiments VS Systems

Experiment	System
Controlled Inputs	Unbounded Inputs
Reversible	Hard to Rollback
Failure is Informative	Failure is costly
Optimized for Insight	Optimized for reliability



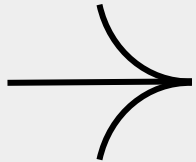
“Systems lie at the end of curiosity and the beginning of responsibility”

2.A. Users see Outliers



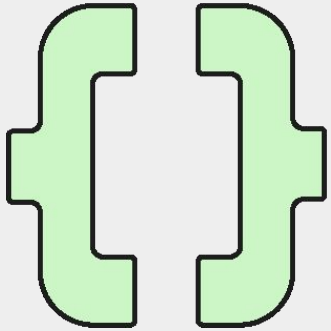
- Metrics **summarize behavior**
- Users experience **single outcomes**
- **Rare failures matter more** than frequent successes
- Scale turns **“rare” into “inevitable”**

Metrics **compress variance into central measures**. That does not mean those cases do not exist



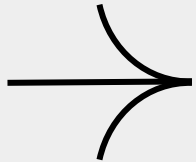
“Systems don’t get judged by their mean. They get judged by their worst memorable failure.”

2.B. Offline Performance ≠ Online Reliability



- Validation sets are curated
- Real **inputs are messy**
- **Assumptions leak** at deployment
- **Confidence scales faster than correctness**

Offline Performance makes ideal assumptions. In deployment, there are no stable distributions and stable input entropies.



“A bad test case can be reasoned about, a bad customer experience is lasting”

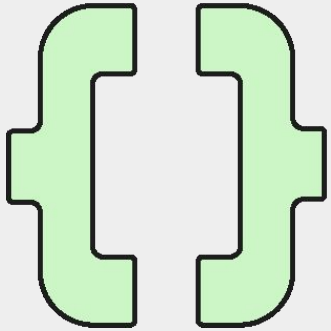
2.C. What Metrics are?

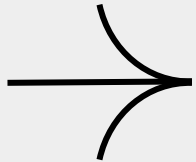
Metrics are guide rails. They are not

- guarantees
- contracts
- truth

They are:

- Early warning signals
- Regression detectors
- Proofs of correctness



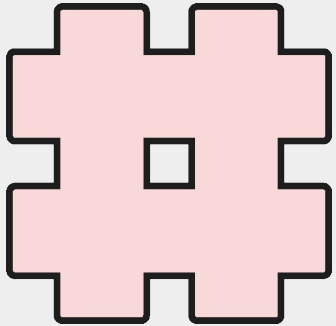


“Metrics help you notice change. They don’t tell you if that change matters.”

3.A. Correctness ≠ Reliability

Correctness is a measure of the models ability to produce correct outputs. In deployment, we often **trade correctness for reliability**, which often proves a better evaluation metric for scalable systems.

- **Correctness**: individual outputs
- **Reliability**: behavior over time
- Users experience **patterns, not samples**
- One-off brilliance is not useful





“Users don’t need perfection. They need predictability.”

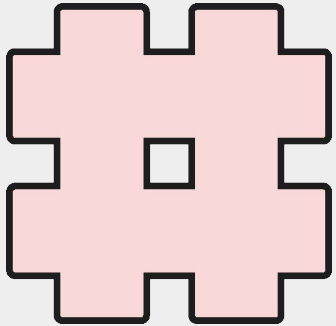
3.B. Failure should be predictable

Seeing as **failure is inevitable, it should be predictable**.

- **All models fail**
- Unpredictable **failure erodes trust**
- Predictable limits enable safe use
- Evaluation maps failure boundaries

Users observe behaviour and emulate input to fit that behaviour.
Making the boundaries clear enforces user trust.

“A good model teaches you how not to use it.”

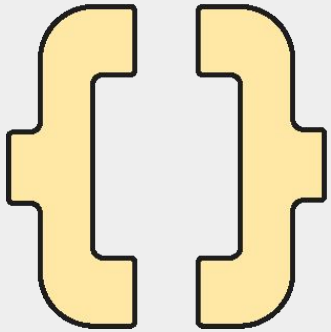




4.A. An Amazonian Disaster

Amazon built an internal ML system to **automatically rank résumés** for technical roles.

- Input: résumé text
- Output: score / ranking
- Training data: historical hiring decisions



Offline metrics (what looked good)

- High agreement with historical hiring outcomes
- Strong accuracy on validation data
- Stable performance during internal testing

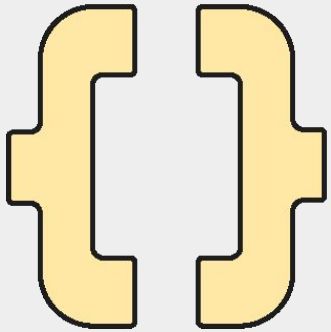
From a pure supervised-learning perspective, the model was *correct*.

4.A. An Amazonian Disaster

What failed in the real world

The model **systematically penalized résumés associated with women**:

- Downgraded résumés containing words like “women’s chess club”
- Preferred patterns historically associated with male candidates



Why this happened

- **Training data encoded past bias**
- The metric optimized “match historical decisions,” not “hire fairly”
- The model was *correct relative to labels*, but **wrong relative to values**

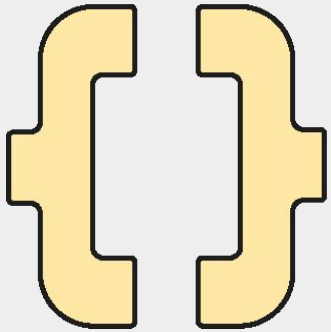
4.B. Zillow Zestimate

What the system was

Zillow used ML models to:

- Predict home prices
- Automatically buy houses (iBuying)

It was **directly connected to capital**.



Offline success

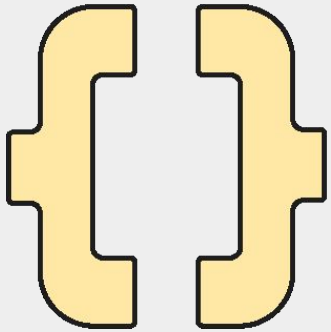
- Zestimate had strong predictive metrics historically
- Small average errors on test data

4.B. Zillow Zestimate

What went wrong

- Housing market dynamics shifted rapidly
- Model assumptions broke under **distribution shift**
- Errors compounded at scale because decisions were automated

The model was “slightly wrong” millions of times.



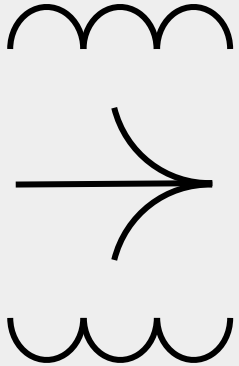
Outcome

- Zillow shut down its iBuying program
- ~25% of workforce laid off
- Billions in losses

5.A. Evaluation as a Process

Evaluation is **not a single step** which happens once after training. It happens at each phase and answers different questions at different phases:

- **Before training:** Establish Baselines
- **During training:** Catch Overfitting
- **After training:** Measure task behaviour
- **After deployment:** Detect drift & regressions



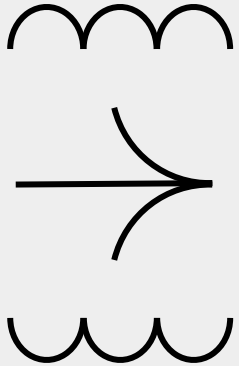
Evaluation at each stage acts as a sanity check for **whether the model still aligns with our end goal**.

5.B. Continuous Evaluation

Every change in the codebase poses risk. Therefore, **evaluation comes as a necessary add-on after each step** in the pipeline to enforce quality.

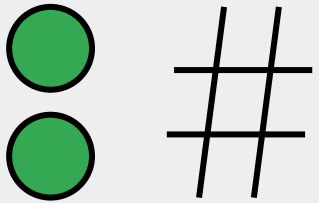
- **New data ≈ new code**
- **Metrics track change, not truth**
- **Humans close the loop**

“Evaluation does not tell you when to stop. It tells you what you broke”



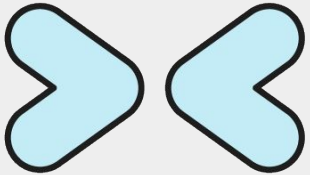
6.A. Metrics take the lead

Although not the entire story, **metrics define the next steps** in the process of improvement of our system.



We need

- Signals
- Comparisons
- Regression detection
- Scale.
- Metrics show us exactly that.

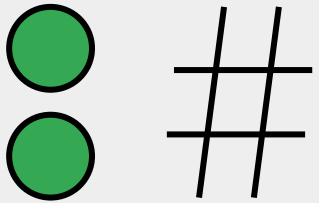


“Metrics are instruments to navigate change in our systems”

6.B. Task specific metrics

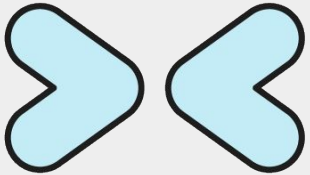
Classic **non-generative tasks usually have hard truths that model output can be put against**. For example:

- Classification → “Is this label correct?”
- Structured output → “Does this match a spec?”



But in generative models, there is no single “truth”

- Generation → “Is this useful to a human?”



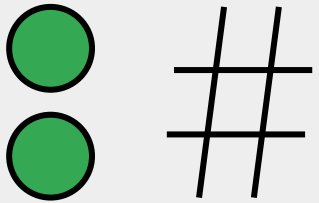
Therefore, **especially in generative tasks, we exercise caution.**

- Closed output spaces → strong, precise metrics
- Open output spaces → weak, proxy

6.C. Common metrics

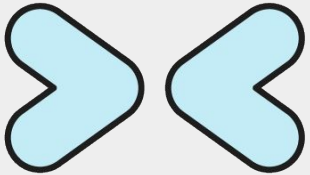
Some common metrics we may find useful in generative tasks are:

- **Accuracy / F1** → label agreement
- **BLEU / ROUGE** → surface similarity
- **Perplexity** → fluency, not correctness



But each one needs to be used cautiously to draw right conclusions.

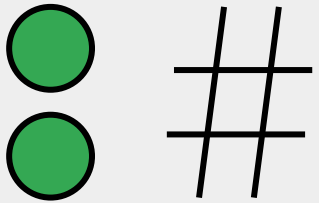
- BLEU rewards overlap, not intent
- ROUGE favors verbosity in some cases
- Low perplexity means “sounds likely,” not “is right”



6.D. Common metrics

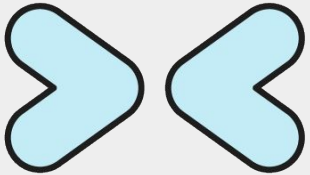
We also have behavior-centric metrics like:

- Consistency over time
- Failure rate per user/session
- Output variance under small input perturbations



These determine *reliability*.

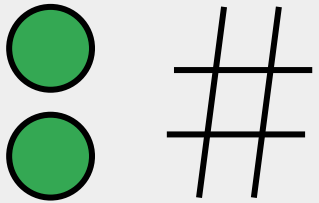
Humans are also an **active part** of the evaluation process.
Systematic audits save systems from future failures.



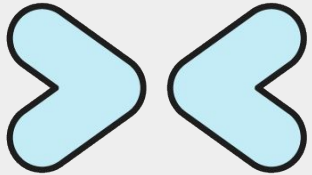
6.E. Post-Deployment

Even after the system is online:

1. Inputs like user language evolve (**data drift**)
2. Semantics of predictions evolve (**concept drift**)
3. Outcomes evolve (**proxy drift**)

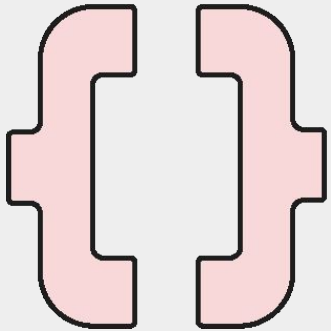


Humans are not stationary creatures. They *evolve* rapidly and thus a **good system evolves** alongside to retain usefulness.



“Metrics help you notice change. They don’t tell you if that change matters.”

7.A. From Model to Artifact



Deploying a Fine Tuned model can take multiple forms:

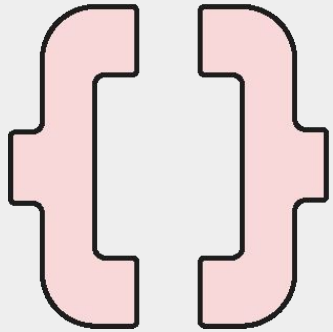
1. API-based deployment (most common)
2. Cloud-managed deployment
3. On-device / edge deployment
4. Batch or offline deployment

In any case, the task is not over after deployment. C**ontinuous monitoring** is a part of the pipeline.

7.B. API-based deployment

You wrap the model in a web service. It works as following:

- Model runs on a server
- Clients send requests (HTTP/REST or gRPC)
- Server returns predictions



Common tools

- FastAPI / Flask (Python)
- TorchServe, TensorFlow Serving
- Docker for packaging

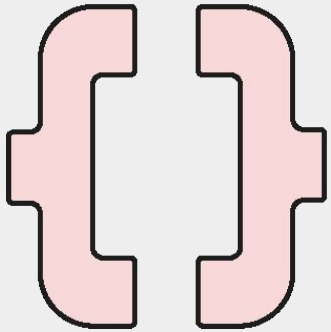
Best for

- Web apps
- Mobile apps
- Multiple users

7.C. Cloud deployment

Cloud providers handle infrastructure and scaling.

- AWS SageMaker
- Google Vertex AI
- Azure ML
- Managed LLM platforms (for fine-tuned LLMs)



Pros

- Auto-scaling
- Monitoring and logging
- Less ops work

Cons

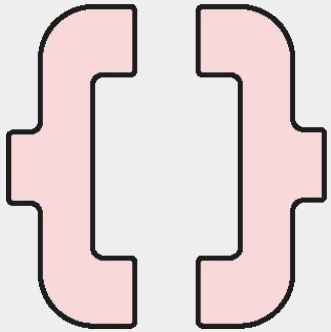
- Cost
- Vendor lock-in

7.D. On-device deployment

The model runs locally (no server).

Examples

- Mobile apps
- Desktop software
- Embedded systems



Techniques

- Model quantization
- Conversion (ONNX, Core ML, TensorRT)
- Smaller architectures

Best for

- Offline usage
- Privacy-sensitive data

7.E. Batch deployment

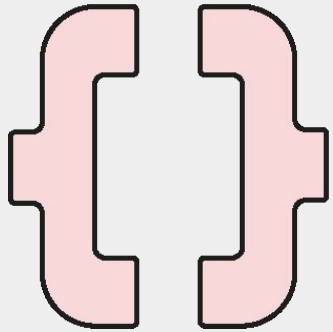
The model runs on scheduled jobs instead of real-time requests.

Examples

- Nightly predictions
- Data pipelines
- Analytics workflows

Tools

- Cron jobs
- Airflow
- Spark
- Cloud batch services



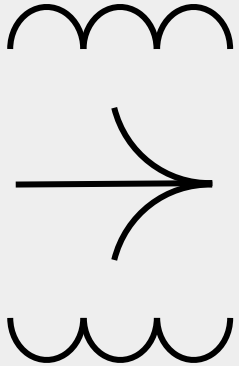
8.A. Failure, Responsibility & Trust

With all this, your **model is now a system** that now faces users.
As we discussed earlier:

- Failure is eminent
- Continuous monitoring and fixes are important

The cost of failure has gone from **“increase in loss” and “a funny gimmick” to losing customer trust**.

Being **receptive to failure** and **responding responsibly** is key to building a good product and **gaining the trust** of your users.



8.B. Negotiable Failure

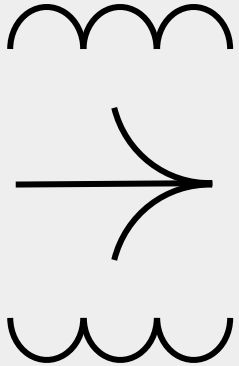
We can bound the failures of our system within desirable confines by design.

Random failure:

- Appears arbitrary
- Breaks user mental models
- Destroys trust quickly

Predictable failure:

- Clear boundaries
- Consistent refusal modes
- Stable degradation



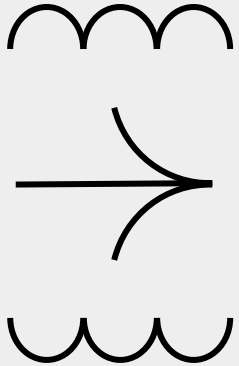
> “A good model teaches you how not to use it.”

8.C. In case of Failure

Once deployed, the **model is no longer “just code.”**

The effect of a **failure can be mitigated** using a variety of techniques such as:

- Versioned deployments
- Feature flags
- Instant rollback
- Rate limiting
- Incident logging



Quiz

Closing Thoughts



Feedback Form

Google Developer Group
Chandigarh University

Thank You!

Model Performance & Deployment - End

